

POP UI DESIGN SYSTEM

Codebase Metadata Documentation

For AI-Powered Figma-to-Code Automation

Package: `com.pop.components.ds_components`
Language: Kotlin | Framework: Jetpack Compose | Platform: Android
Generated: April 10, 2026

Table of Contents

1. Executive Summary & Purpose
2. Architecture Overview
3. Design Token System (Theme Layer)
4. Complete Component Registry
5. Component Deep-Dive: API Signatures & Properties
6. File Dependency Graph & Linkages
7. How to Find Any Component's Code (AI Lookup Guide)
8. Figma-to-Code Mapping Strategy
9. Code Patterns & Conventions
10. Preview / Testing System
11. Utility & Effect Systems
12. Appendix: Full File Index

1. Executive Summary & Purpose

This document serves as the complete metadata reference for the POP UI Design System codebase. It is designed specifically to give an AI agent full context to:

- (a) Understand the codebase structure, patterns, and conventions used across all components.
- (b) Quickly locate any component's code given a Figma design element name or visual description.
- (c) Map Figma design system properties (colors, spacing, typography, radii) to their code equivalents.
- (d) Generate new component code that is consistent with existing patterns when a component doesn't yet exist.
- (e) Aggregate individual component code into complete screen layouts matching Figma designs.

Codebase Statistics

Metric	Value
Total Kotlin Files	104 files (78 components + 26 previews)
Total Codebase Size	~1.6 MB of Kotlin source
Main Component Files	58 files in ds_components/
Scrim/Blur Subsystem	20 files in ds_components/scrim/
Preview/Test Files	26 files in preview/
Primary Package	com.pop.components.ds_components
UI Framework	Jetpack Compose (pure Compose, no XML layouts)
Design System	POP DS — Figma-aligned tokens
Largest Component	PopInputFieldV2.kt (108 KB, ~2500 lines)
Component Naming	Pop[ComponentName] prefix convention

2. Architecture Overview

2.1 Directory Structure

```
mnt/
├── ds_components/                # Main component library (58 files)
│   ├── Pop*.kt                  # UI components (@Composable functions)
│   ├── Button*.kt               # Button enums, specs, states
│   ├── AvatarType.kt            # Avatar sealed interface + AvatarSize enum
│   ├── SquircleShape.kt         # Custom Figma-style corner shape
│   ├── EmptyStateComponent.kt   # Empty state pattern
│   ├── Scrim.kt                 # Top-level scrim composable
│   └── scrim/                   # Blur effect subsystem (20 files)
│       ├── ScrimCore.kt         # Core state & API
│       ├── BlurModifierExtensions.kt # Modifier extensions
│       └── Scrim*.kt            # Render effects, delegates, nodes
└── preview/                     # Preview composables (26 files)
    └── Pop*Previews.kt         # @Preview annotated test composables
```

2.2 Package Dependency Architecture

The codebase follows a strict layered architecture with clear dependency direction:

Layer 1 — Theme / Design Tokens (`com.pop.components.theme`)

This is the foundation layer. It defines all design tokens: `PopColor`, `PopTypography`, `PopSpacing`, `PopRadius`, `PopStroke`, `PopGradient`, `Icons`, `IconColor`, `TextColor`, `SurfaceColor`, `BorderColor`, `PopShapes`, `FlashGrid`, etc. All components depend on this layer. The AI must map Figma variables/tokens to these theme objects.

Layer 2 — Models / Configs (`com.pop.components.models`)

Data classes and enums that configure components: `PopInputFieldConfig`, `PopTopBarConfig`, `PopBottomSheetConfig`, `PopChipConfig`, `PopOtpFieldConfig`, `AccordionConfig`, `VisualElement`, `InputFieldStatus`, `PopInputFieldType`, `ButtonSize`, etc. These carry Figma property mappings.

Layer 3 — Utilities (`com.pop.components.utils`)

Modifier extensions and helper functions: `popBackground()`, `popBorder()`, `glowEffect()`, `intenseGlowEffect()`, `noiseOverlay()`, `horizontalGradientFade()`, `bottomPrimaryGradientWithDither()`. These translate Figma effects (glows, gradients, noise) into Compose modifiers.

Layer 4 — Shared Compose Components (`com.pop.components.compose_components`)

Low-level shared composables: `PopDivider`, `PopDividerStyle`, and resource references (R class).

Layer 5 — DS Components (com.pop.components.ds_components) — THIS CODEBASE

The main component library. Every file in ds_components/ and ds_components/scrim/ lives here. These are the @Composable functions that the AI will search, retrieve, and assemble.

Layer 6 — Animation Utilities (com.pop.components.animation.utils)

Animation helpers: rootContentView(), runScaleDownAnimation(), runResetScaleAnimation(). Used by PopBottomSheet for background scale-down effects.

3. Design Token System (Theme Layer)

Every component references the centralized theme package. The AI MUST understand these tokens to correctly map Figma properties to code. Below is the complete token mapping:

3.1 Color Tokens

Theme Object	Import Path	Purpose	Example Tokens
PopColor	com.pop.components.theme.PopColor	Primary color palette	PopColor.WhiteBlack, PopColor.Brand.Brand500
TextColor	com.pop.components.theme.TextColor	Text color variants	TextColor.Primary, .PrimaryInvert, .Disabled, .Brand, .Destructive, .Success, .Warning, .Tertiary, .PrimaryTransparent40
SurfaceColor	com.pop.components.theme.SurfaceColor	Background/surface fills	SurfaceColor.Primary, .PrimaryInvert, .PrimaryDisabled70, .SecondaryDisabled, .PrimaryTransparent50, .SecondaryFromTokens
IconColor	com.pop.components.theme.IconColor	Icon tint colors	IconColor.PrimaryFromTokens, .PrimaryInvert, .Disabled, .BrandFromTokens, .DestructiveFromTokens, .SuccessFromTokens
BorderColor	com.pop.components.theme.BorderColor	Border/stroke colors	BorderColor.Secondary, .Tertiary, .SecondaryFromTokens, .PrimaryInvertTransparent10, .PrimaryTransparent30
AvatarColor	com.pop.components.theme.AvatarColor	Avatar-specific colors	AvatarColor.PersonBorder, .NonPersonBackground, .NonPersonBorder
PrimaryColor	com.pop.components.theme.PrimaryColor	Core primary colors	Used directly in components
PopColors	com.pop.components.theme.PopColors	Extended color set	Used by PopAccordion

3.2 Typography Tokens

Token	Import	Details
PopTypography.figtree	com.pop.components.theme.PopTypography	Primary font family (Figtree) — used across ALL components for text rendering

FontWeight(500)	Standard Kotlin	Medium weight — used for body text, labels, buttons
FontWeight(600)	Standard Kotlin	SemiBold weight — used for large buttons, headings
FontWeight.Bold	Standard Kotlin	Bold weight — used for avatar initials, emphasis
Label sizes	ButtonSpecs.FontSize	XSmall: 12sp, Small: 14sp, Medium: 16sp, Large: 18sp
Line heights	ButtonSpecs.LineHeight	XSmall: 16sp, Small: 20sp, Medium: 24sp, Large: 24sp

3.3 Spacing & Dimension Tokens

Token Object	Import	Values / Purpose
PopSpacing	com.pop.components.theme.PopSpacing	Discrete spacing values — used for padding, margins, gaps between elements
PopRadius	com.pop.components.theme.PopRadius	Corner radius tokens: Large (16dp), XLarge (20dp), XLLarge (999dp for pill)
PopStroke	com.pop.components.theme.PopStroke	Border stroke widths: Default = 0.5dp
PopShapes	com.pop.components.theme.PopShapes	Predefined shape instances
FlashGrid	com.pop.components.theme.FlashGrid	Responsive grid system for layout
ButtonSpecs.Spacing	In ButtonSpecs.kt	S0(0), S4(4), S6(6), S8(8), S10(10), S12(12), S16(16), S28(28), S36(36), S44(44), S56(56)
ButtonSpecs.Height	In ButtonSpecs.kt	XSmall: 32dp, Small: 36dp, Medium: 44dp, Large: 56dp
ButtonSpecs.Padding	In ButtonSpecs.kt	XSmall: 12dp, Small: 16dp, Medium: 20dp, Large: 28dp
ButtonSpecs.IconSize	In ButtonSpecs.kt	XSmall: 16dp, Small: 18dp, Medium: 20dp, Large: 24dp
ButtonSpecs.CornerRadius	In ButtonSpecs.kt	XSmall: 16dp, Small: 18dp, Medium: 22dp, Large: 28dp

3.4 Gradient Tokens

Token	Import	Usage
PopGradient	com.pop.components.theme.PopGradient	Base gradient type — PopGradient.Linear(colors, angle) and PopGradient.Solid(color)
GradientPreset	com.pop.components.theme.GradientPreset	Predefined gradient presets: ButtonBrandPrimaryLargeHorizontal, ButtonPrimaryInvertLarge, ButtonSecondaryLargeHorizontal,

		ButtonSuccessLarge, ButtonDestructiveLarge, ButtonWhiteLarge, ButtonPrimaryDisabled, ButtonBrandPrimaryDisabled, GradientOrange, GradientGreen, GradientRed, BadgeWhitePrimary, SurfacePrimary, AvatarBrand, etc.
SurfaceColor.Gradient	com.pop.components.theme.SurfaceColor	Surface gradient tokens: SurfaceColor.Gradient.Secondary.Start/End, SurfaceColor.Gradient.Brand.Start/End

3.5 Icon System

Token	Import	Details
Icons	com.pop.components.theme.Icons	Type-safe icon name enum: Icons.Check, Icons.Cross, Icons.ChevronLeft, Icons.Share05, Icons.Filter, Icons.Search, etc.
IconName	com.pop.components.theme.IconName	Type alias for icon identification
IconStyle	com.pop.components.theme.IconStyle	Icon style variants: IconStyle.Outline (primary), IconStyle.Solid
IconSize	com.pop.components.theme.IconSize	Predefined icon size tokens
PopIconResources	com.pop.components.theme.PopIconResources	Icon resolution: PopIconResources.getIconResourceId(iconName, style) -> drawable res ID
PopIconConfig	com.pop.components.theme.PopIconConfig	Icon configuration holder
PopIcons	com.pop.components.theme.PopIcons	Icon size definitions

4. Complete Component Registry

This is the master list of every component in the design system. The AI should use this as the primary lookup table when matching Figma elements to code.

4.1 Buttons & Actions

Component	File	Figma Node	Description
PopButtonV2	PopButtonV2.kt	1248-823	Primary button with label + optional left/right icons. Variants: Primary, Secondary, Tertiary, BrandPrimary, White, Ghost, BrandGhost. States: Active, Disabled, Loading, Destructive. Sizes: XSmall, Small, Medium, Large.
PopVerticalButton	PopVerticalButton.kt	—	Vertically stacked icon + label button
ButtonVariant	ButtonVariant.kt	—	Enum: Primary, Secondary, Tertiary, BrandPrimary, White, Ghost, BrandGhost
ButtonState	ButtonState.kt	—	Enum: Active, Disabled, Loading, Destructive
ButtonLoadingState	ButtonLoadingState.kt	—	Enum: Default, Success, Destructive, Intermediate
ButtonSpecs	ButtonSpecs.kt	1248-823	Exact Figma specs: heights, paddings, font sizes, icon sizes, corner radii, colors, gradients

4.2 Form Controls & Inputs

Component	File	Figma Node	Description
PopInputFieldV2	PopInputFieldV2.kt	—	Multi-type input field. Types: Basic, Mobile, Search, Small, UnderlineNakedSmall, UnderlineNakedLarge. Status: Error, Info, Success, Warning. Largest component (108KB).
PopOtpField	PopOtpField.kt	—	OTP/PIN entry field with individual digit boxes, auto-advance, paste support
PopCheckBoxV2	PopCheckBoxV2.kt	—	Checkbox with custom styling matching Figma DS
PopRadio	PopRadio.kt	—	Radio button component
PopToggle	PopToggle.kt	5345-4180	Toggle switch. Sizes: Large, Medium. States: On, Off, Indeterminate. Supports promoted (brand) variant.
PopKeyPad	PopKeyPad.kt	—	Numeric keypad for PIN/amount entry
PopPills	PopPills.kt	4575-1506 5	Pill-shaped selectable group (2 or 3 pills). Selection: Left, Center, Right.

PopInputFieldHelpers	PopInputFieldHelpers.kt	—	Helper functions for input field logic
PopKeyPadHelpers	PopKeyPadHelpers.kt	—	Helper functions for keypad logic

4.3 Navigation & Layout

Component	File	Figma Node	Description
PopTopBar	PopTopBar.kt	5435-4761	Complete top bar with app bar, optional merchant image, title bar. Scroll-aware collapsing with blur.
PopTopBarWithContent	PopTopBarWithContent.kt	—	Extended top bar variant with custom content sections (49KB)
PopTopBarWithCustomContent	PopTopBarWithCustomContent.kt	—	Top bar with fully custom content area
PopTopBarWithXmlContent	PopTopBarWithXmlContent.kt	—	Top bar bridging XML-based content
PopAppBar	PopAppBar.kt	—	Standalone app bar component (navigation icon + action slots)
PopAppBarDsl	PopAppBarDsl.kt	—	DSL for configuring app bar: ConfigureAppBar() composable with slot1(), slot2() builder
PopAppBarManager	PopAppBarManager.kt	—	State management for app bar across screens
PopBottomBar	PopBottomBar.kt	—	Bottom navigation bar with animated selection, end sticky item, glow effects
PopBottomSheet	PopBottomSheet.kt	—	Modal bottom sheet with drag handle, scale-down animation, gradient backgrounds
PopTabLayout	PopTabLayout.kt	3018-1630	Underline-style tab layout. Width modes: Equal (distributed), Wrap (scrollable). Swipeable.
PopTitleBar	PopTitleBar.kt	—	Title bar sub-component used within PopTopBar
PopStickyHeader	PopStickyHeader.kt	—	Sticky header for scrollable lists
PopStickyHeaderGrid	PopStickyHeaderGrid.kt	—	Sticky header variant for grid layouts
PopLazyColumn	PopLazyColumn.kt	—	Enhanced LazyColumn with PopTopBar integration
PopLazyVerticalGrid	PopLazyVerticalGrid.kt	—	Enhanced LazyVerticalGrid with PopTopBar integration
PopTopBarScrollBehavior	PopTopBarScrollBehavior.kt	—	Scroll behavior state/config for collapsible top bars

4.4 Content Display

Component	File	Figma Node	Description
PopListItem	PopListItem.kt	405-10994	Generic list item: headline, supporting text, leading avatar, trailing content. Supports marquee, loaders, dividers, underlines.
PopChip	PopChip.kt	1526-12162	Chip component. Variants: Basic, WithClose, WithDropdown. Modes: Toggleable, Static.
PopChipStack	PopChipStack.kt	—	Container for arranging multiple chips
PopBadge	PopBadge.kt	5529-3928	Badge with label + optional icons. Types: Orange, Green, Red, WhitePrimary, WhiteTertiary, Warning. Supports bg + glow.
PopAccordion	PopAccordion.kt	—	Expandable/collapsible accordion with bounce animation
PopCarousel	PopCarousel.kt	—	Horizontal carousel with pagination, auto-scroll, gradient fades
PopHeroCarousel	PopHeroCarousel.kt	—	Full-width hero carousel variant
PopDot	PopDot.kt	—	Dot indicator component (for carousels, pagination)
EmptyStateComponent	EmptyStateComponent.kt	—	Empty state pattern: image + title + subtitle + optional button
PopOfferHighlight	PopOfferHighlight.kt	—	Offer/promotion highlight card
PopOfferToggle	PopOfferToggle.kt	—	Toggle specifically for offer/promotion display
PopToastView	PopToastView.kt	—	Toast notification component

4.5 Avatar System

Component	File	Figma Node	Description
PopAvatar	PopAvatar.kt	856-171	Generic avatar. Types (sealed interface): People, Favorite, Brand, Icon, Illustration, Image, RawImage. Sizes: XXLLarge(130), XLarge(64), Large(56), Medium(36), Small(28), XSmall(16).
AvatarType	AvatarType.kt	—	Sealed interface: People, Favorite, Brand, Icon, Illustration, Image, RawImage. Each with properties for background, border, shape.
AvatarSize	AvatarType.kt	5568-9341	Enum with size values, initials style, logo padding, corner radius, icon size per variant
OverlappingAvatarIcons	OverlappingAvatarIcons.kt	—	Overlapping avatar group display

FavIcon	FavIcon.kt	—	Favorite icon overlay for avatars
---------	------------	---	-----------------------------------

4.6 Text & Animation

Component	File	Description
PopMarqueeText	PopMarqueeText.kt	Horizontally scrolling marquee text with configurable speed, delay, loop count, gradient fades
AlternatingSubtitleText	AlternatingSubtitleText.kt	Text that alternates between different subtitle strings
RotatingText	RotatingText.kt	Text that rotates through different values with animation
PopVisualElement	PopVisualElement.kt	Universal renderer for VisualElement (Image, Rive animation, Color). Uses Coil for async images.
RiveAnimationCompose	RiveAnimationCompose.kt	Rive animation integration for Jetpack Compose

4.7 Loading & Skeleton

Component	File	Figma Node	Description
PopShimmer	PopShimmer.kt	8220-35278	Shimmer loader. Types: People, Brand, List, Payment list, Body, Title, Table item, Card, CartContent, AddressItem. Multiple size variants per type.
PopShimmerBlocks	PopShimmerBlocks.kt	—	Pre-composed shimmer block layouts for common loading states

4.8 Shape & Effects

Component	File	Description
SquircleShape	SquircleShape.kt	Figma-style smooth rounded rectangle using superellipse Bezier curves. Params: cornerRadius (Dp?), smoothing (0-1, default 0.6). Used by PopBadge and others.
Scrim	Scrim.kt	Top-level blur/scrim composable for background blur effects
scrim/ subsystem	scrim/*.kt (20 files)	Complete blur effect engine: ScrimCore, ScrimState, ScrimEffect, ScrimBlurRenderEffect, ScrimEffectNode, BlurModifierExtensions, ScrimProgressive, etc. Adapted from Haze library.

5. Component Deep-Dive: API Signatures & Properties

This section documents the `@Composable` function signatures for the most commonly used components. The AI should use these signatures when generating code or mapping Figma properties.

5.1 PopButtonV2

```
@Composable
fun PopButtonV2(
    text: CharSequence,
    modifier: Modifier = Modifier,
    variant: ButtonVariant = ButtonVariant.Primary,
    state: ButtonState = ButtonState.Active,
    buttonLoadingState: ButtonLoadingState? = null,
    size: ButtonSize = ButtonSize.Large,
    leftIcon: VisualElement? = null,
    rightIcon: VisualElement? = null,
    widthType: ButtonWidthType = ButtonWidthType.Wrap,
    onDisabledClick: (() -> Unit)? = null,
    onClick: () -> Unit
)
```

5.2 PopBadge

```
@Composable
fun PopBadge(
    modifier: Modifier = Modifier,
    label: String,
    type: BadgeType,           // Orange, Green, Red, WhitePrimary, WhiteTertiary, Warning
    hasBg: Boolean = true,
    hasGlow: Boolean = false,
    leftIcon: IconName? = null,
    rightIcon: IconName? = null
)
```

5.3 PopToggle

```
@Composable
fun PopToggle(
    checked: Boolean,
    onCheckedChange: (Boolean) -> Unit,
    modifier: Modifier = Modifier,
    size: PopToggleSize = PopToggleSize.Large,    // Large, Medium
)
```

```

    promoted: Boolean = false,                // Orange brand glow
    enabled: Boolean = true,
    indeterminate: Boolean = false
)

```

5.4 PopAvatar

```

@Composable
fun PopAvatar(
    modifier: Modifier = Modifier,
    type: AvatarType,      // People, Favorite, Brand, Icon, Illustration, Image, RawImage
    size: AvatarSize,      // XXLarge(130dp), XLarge(64), Large(56), Medium(36), Small(28),
    XSmall(16)
    contentDescription: String? = null,
    isDisabled: Boolean = false,
    topRightComposable: @Composable (() -> Unit)? = null
)

```

5.5 PopListItem

```

@Composable
fun PopListItem(
    headlineText: String,
    modifier: Modifier = Modifier,
    supportingText: String? = null,
    leadingAvatar: @Composable (() -> Unit)? = null,
    trailingContent: @Composable (() -> Unit)? = null,
    onClick: (() -> Unit)? = null,
    enabled: Boolean = true,
    showDivider: Boolean = true,
    // ... 30+ additional parameters for marquee, icons, loaders, underlines
)

```

5.6 PopChip

```

@Composable
fun PopChip(
    // Uses PopChipConfig from com.pop.components.models
    variant: PopChipVariant, // Basic, WithClose, WithDropdown
    mode: PopChipMode,       // Toggleable, Static
    // ... additional config parameters
)

```

5.7 PopTabLayout

```

@Composable

```

```

fun PopTabLayout(
    tabs: List<PopTabItem>,
    selectedTabId: String,
    onTabSelected: (String) -> Unit,
    modifier: Modifier = Modifier,
    widthMode: PopTabLayoutWidth = PopTabLayoutWidth.Wrap, // Equal, Wrap
    colors: PopTabLayoutColors = PopTabLayoutDefaults.colors(),
    enableSwipe: Boolean = true,
    swipeThreshold: Float = 50f,
    showDivider: Boolean = false
)

```

5.8 PopTopBar

```

@Composable
fun PopTopBar(
    modifier: Modifier = Modifier,
    config: PopTopBarConfig = PopTopBarConfig.default(),
    scrollState: ScrollState? = null,
    lazyListState: LazyListState? = null,
    scrollBehavior: PopTopBarScrollBehaviorState? = null,
    scrollBehaviorConfig: PopTopBarScrollBehaviorConfig = ...,
    contentNeedsScrolling: Boolean = true,
    centerRightSlot: (@Composable () -> Unit)? = null,
    scrimState: ScrimState? = null,
    isContentScrollingBehind: Boolean = false
)

```

5.9 PopShimmer

```

@Composable
fun PopShimmer(
    modifier: Modifier = Modifier,
    type: LoaderType = LoaderType.People, // People, Brand, List, Payment, Body, Title,
    ...
    size: LoaderSize = LoaderSize.Small128 // Multiple size variants per type
)

```

5.10 EmptyStateComponent

```

@Composable
fun EmptyStateComponent(
    imageResId: Int,
    title: String,
    modifier: Modifier = Modifier,
    subtitle: String? = null,

```

```
buttonText: String? = null,  
onButtonClick: (() -> Unit)? = null,  
buttonVariant: ButtonVariant = ButtonVariant.Secondary,  
buttonState: ButtonState = ButtonState.Active,  
buttonSize: ButtonSize = ButtonSize.Medium,  
buttonIcon: VisualElement? = null  
)
```


6. File Dependency Graph & Linkages

This section maps how files depend on each other. The AI needs this to understand which files to read when assembling a complete screen.

6.1 Core Dependency Chains

PopButtonV2.kt depends on:

```
ButtonVariant.kt → ButtonState.kt → ButtonLoadingState.kt → ButtonSpecs.kt
Theme: PopColor, PopGradient, PopTypography, TextColor, SurfaceColor, IconColor,
BorderColor
Models: VisualElement, ButtonSize (from com.pop.components.models)
Utils: noiseOverlay, intenseGlowEffect, popBackground, popBorder
Icons: Icons, IconStyle (from theme)
```

PopTopBar.kt depends on:

```
PopAppBar.kt → PopAppBarDsl.kt → PopAppBarManager.kt
PopTitleBar.kt
PopTopBarScrollBehavior.kt → PopTopBarScrimType.kt
scrim/ScrimState, scrim/blurModifier, scrim/rememberScrimState
Models: PopTopBarConfig, MerchantImageSize, StatusBarTheme
Compose: PopDivider (from compose_components)
```

PopInputFieldV2.kt depends on:

```
PopInputFieldHelpers.kt
Models: PopInputFieldConfig, PopInputFieldType, InputFieldStatus
Models: BasicInputFieldConfig, MobileInputFieldConfig, SearchInputFieldConfig
Models: SmallInputFieldConfig, UnderlineNakedLargeConfig, UnderlineNakedSmallConfig
Theme: Full token set (PopColor, PopTypography, PopRadius, PopSpacing, PopStroke, etc.)
Utils: glowEffect, horizontalGradientFade
```

PopAvatar.kt depends on:

```
AvatarType.kt (sealed interface + AvatarSize enum — SAME FILE)
PopVisualElement.kt → VisualElement model
Theme: BorderColor, GradientPreset, IconColor, PopGradient, PopIconResources, PopColor,
SurfaceColor
Utils: popBackground, popBorder
```

PopBottomSheet.kt depends on:

```
Models: PopBottomSheetConfig, BottomSheetBackgroundType, BottomSheetGradientShape,
VisualElement
Theme: BorderColor, GradientPreset, Icons, IconStyle, PopStroke, SurfaceColor, TextColor
Animation: rootContentView, runScaleDownAnimation, runResetScaleAnimation
```

Utils: popBackground

PopListItem.kt depends on:

Compose: PopDivider, PopDividerStyle

Models: VisualElement

Theme: PopTypography, TextColor, SurfaceColor, BorderColor

Optional: PopMarqueeText (when marquee enabled), PopShimmer (when loaders enabled)

6.2 Cross-Component Dependencies

Component Used By	Used In
PopVisualElement	PopAvatar, PopCarousel, PopHeroCarousel, PopListItem, EmptyStateComponent
PopButtonV2	EmptyStateComponent, PopBottomSheet (action buttons)
PopBadge	PopListItem (trailing content), PopCarousel items
PopDivider	PopListItem, PopTabLayout, PopTopBar, PopAccordion
PopShimmer	PopListItem (loader states), standalone skeleton screens
PopMarqueeText	PopListItem (headline/supporting text marquee), PopBottomBar
SquircleShape	PopBadge, PopAvatar (Brand type), general purpose
PopAppBarManager	PopAppBarDsl, PopAppBar — manages state across screens
ScrimState/blurModifier	PopTopBar, PopTopBarWithContent, Scrim.kt
PopToggle	PopListItem trailing content, PopOfferToggle
PopChip/PopChipStack	Filter bars, PopTopBar content areas
PopTitleBar	PopTopBar, PopTopBarWithContent, PopTopBarWithCustomContent

7. How to Find Any Component's Code (AI Lookup Guide)

This is the critical section for the AI agent. Follow this decision tree to locate any component's code from a Figma design:

Step 1: Identify the Figma Element Type

Figma Element	Maps To	File to Read
Button (any variant)	PopButtonV2	PopButtonV2.kt + ButtonSpecs.kt + ButtonVariant.kt
Input / Text field / Search bar	PopInputFieldV2	PopInputFieldV2.kt (check config type: Basic, Mobile, Search, Small, Underline)
Toggle / Switch	PopToggle	PopToggle.kt
Checkbox	PopCheckBoxV2	PopCheckBoxV2.kt
Radio button	PopRadio	PopRadio.kt
Chip / Tag / Filter chip	PopChip	PopChip.kt (variant: Basic, WithClose, WithDropdown)
Chip group / stack	PopChipStack	PopChipStack.kt
Badge / Label / Status indicator	PopBadge	PopBadge.kt (type: Orange, Green, Red, etc.)
Pills / Segmented control	PopPills	PopPills.kt
Avatar / Profile image	PopAvatar	PopAvatar.kt + AvatarType.kt
List item / Row	PopListItem	PopListItem.kt
Top bar / Header / Navigation bar	PopTopBar	PopTopBar.kt + PopAppBar.kt + PopTitleBar.kt
Bottom navigation bar	PopBottomBar	PopBottomBar.kt
Bottom sheet / Modal	PopBottomSheet	PopBottomSheet.kt
Tab bar / Tabs	PopTabLayout	PopTabLayout.kt
Carousel / Slider	PopCarousel	PopCarousel.kt or PopHeroCarousel.kt
Accordion / Expandable	PopAccordion	PopAccordion.kt
Shimmer / Skeleton loader	PopShimmer	PopShimmer.kt + PopShimmerBlocks.kt
Toast / Notification	PopToastView	PopToastView.kt
Empty state	EmptyStateComponent	EmptyStateComponent.kt
OTP / PIN entry	PopOtpField	PopOtpField.kt
Keypad / Number pad	PopKeyPad	PopKeyPad.kt
Scrolling text / Marquee	PopMarqueeText	PopMarqueeText.kt
Offer highlight card	PopOfferHighlight	PopOfferHighlight.kt
Blur / Frost glass effect	Scrim system	Scrim.kt + scrim/*.kt

Image / Icon / Lottie / Rive	PopVisualElement	PopVisualElement.kt + RiveAnimationCompose.kt
Rounded square (iOS-style)	SquircleShape	SquircleShape.kt

Step 2: Read the Component File

Once identified, read the .kt file from `ds_components/`. The component's `@Composable` function signature tells you all available properties. Match each Figma property (variant, size, state, color) to the corresponding parameter.

Step 3: Map Figma Properties to Code

Use the theme token tables (Section 3) to translate Figma design tokens. Example: if Figma shows 'Surface/Primary' background, use `SurfaceColor.Primary`. If it shows 'Text/Brand' color, use `TextColor.Brand`. If it shows 'Border/Secondary' stroke, use `BorderColor.Secondary`.

Step 4: If Component Not Found — Generate New Code

If no existing component matches, the AI should generate new code following the patterns in Section 9. Use the same naming convention (`Pop[Name]`), same import structure, same theme tokens, and place in `ds_components/` package.

8. Figma-to-Code Mapping Strategy

This section defines the complete workflow for converting a Figma design into Kotlin Compose code using this codebase.

8.1 Phase 1: Design Decomposition

Given a Figma design link, the AI should split the design into its constituent components by scanning top-to-bottom, left-to-right:

1. Identify the screen layout structure (Column, Row, Box, LazyColumn, etc.)
2. Identify the top bar / navigation bar type
3. List every distinct UI element: buttons, inputs, avatars, chips, badges, list items, etc.
4. Note each element's properties: variant, size, state, colors, text, icons
5. Identify spacing between elements (map to PopSpacing tokens)
6. Identify any overlay effects: bottom sheets, toasts, blur/scrim

8.2 Phase 2: Component Lookup

For each identified element, use Section 7 (AI Lookup Guide) to find the matching component. Read the component file to get the exact `@Composable` signature and available properties.

8.3 Phase 3: Property Mapping

Map each Figma property to its code equivalent:

Figma Property	Code Equivalent	Example
Variant = Primary	<code>variant = ButtonVariant.Primary</code>	<code>PopButtonV2(variant = ButtonVariant.Primary, ...)</code>
Size = Large	<code>size = ButtonSize.Large</code>	Height 56dp, padding 28dp, fontSize 18sp
State = Disabled	<code>state = ButtonState.Disabled</code>	Applies disabled colors automatically
Fill = Brand gradient	<code>GradientPreset.ButtonBrandPrimaryLargeHorizontal</code>	Applied via <code>popBackground()</code>
Corner radius = 16	<code>PopRadius.Large</code> or <code>RoundedCornerShape(16.dp)</code>	Applied via <code>.clip()</code> or <code>shape</code> param
Stroke = 0.5dp, Secondary	<code>border(PopStroke.Default, BorderColor.Secondary)</code>	Applied via <code>.border()</code> modifier
Font = Figtree Medium 14sp	<code>PopTypography.figtree, FontWeight(500), 14.sp</code>	Text composable parameters
Color = Text/Primary	<code>TextColor.Primary</code>	Used in <code>Text()</code> color parameter

Spacing = 8dp gap	Arrangement.spacedBy(8.dp) or Spacer(8.dp)	Row/Column arrangement
Icon = check, Outline	Icons.Check, IconStyle.Outline	Via VisualElement.buildFrom()
Shadow / Glow	Modifier.glowEffect() or .intenseGlowEffect()	From utils package
Blur / Frost glass	scrim/blurModifier or ScrimState	From scrim subsystem
Noise texture	Modifier.noiseOverlay(NoisePresets.*)	From utils package

8.4 Phase 4: Code Assembly

Assemble the complete screen code by composing components inside a layout structure. Follow this template:

```
@Composable
fun MyScreen() {
    // 1. Top-level layout
    Column(modifier = Modifier.fillMaxSize()) {
        // 2. Top bar (if present)
        PopTopBar(config = PopTopBarConfig.default())

        // 3. Scrollable content
        LazyColumn(modifier = Modifier.weight(1f)) {
            // 4. Individual components
            item { PopListItem(...) }
            item { PopButtonV2(...) }
        }

        // 5. Bottom bar (if present)
        PopBottomBar(...)
    }
}
```

9. Code Patterns & Conventions

9.1 Naming Conventions

Element	Convention	Examples
Component functions	Pop[Name] with @Composable	PopButtonV2, PopAvatar, PopListItem, PopShimmer
Component files	Pop[Name].kt — one file per component	PopButtonV2.kt, PopAvatar.kt
Enum files	Descriptive name matching enum	ButtonVariant.kt, AvatarType.kt
Spec objects	[Component]Specs	ButtonSpecs
Config data classes	Pop[Component]Config	PopTopBarConfig, PopBottomSheetConfig, PopInputFieldConfig
Preview files	Pop[Name]Previews.kt in preview/	PopButtonPreviews.kt, PopAvatarPreviews.kt
Size enums	[Component]Size or Pop[Component]Size	ButtonSize, AvatarSize, PopToggleSize, LoaderSize
Variant enums	[Component]Variant or Pop[Component]Variant	ButtonVariant, PopChipVariant, BadgeType
State enums	[Component]State	ButtonState, PopToggleState, InputFieldStatus
Internal helpers	Private @Composable or extension functions	Badgelcon(), rememberDebounceClick()

9.2 Component Structure Pattern

Every component follows this structural pattern:

```
package com.pop.components.ds_components

// 1. Imports: theme tokens, models, utils, compose foundation
import com.pop.components.theme.*
import com.pop.components.models.*
import com.pop.components.utils.*

// 2. Enums/constants for variants, sizes, states (if component-specific)
enum class MyComponentVariant { A, B, C }

// 3. KDoc with Figma link
/**
 * PopMyComponent - Description
 * Figma: https://www.figma.com/design/yDf8UqgkvEJofpxyCYAmIC/POP-DS?node-id=XXXX-XXXX
 */
```

```

*/
@Composable
fun PopMyComponent(
    modifier: Modifier = Modifier,
    // ... parameters with defaults
) {
    // 4. Derive colors/dimensions from variant + state + theme tokens
    val bgColor = when(variant) { ... }
    // 5. Compose UI tree using Box/Row/Column
    Box(modifier = modifier.clip(shape).popBackground(...)) { ... }
}

```

9.3 Key Patterns the AI Must Follow

Pattern 1: Theme Token Usage (NEVER hardcode colors)

Always use theme tokens: `TextColor.Primary` instead of `Color(0xFFFFFFFF)`, `PopRadius.Large` instead of `16.dp` for standard radii, `PopStroke.Default` instead of `0.5.dp`, `PopTypography.figtree` instead of `FontFamily.Default`.

Pattern 2: Gradient Backgrounds

Use `Modifier.popBackground(gradient = GradientPreset.X.gradient, shape = shape)` from `utils`. Never manually create `Brush.linearGradient` unless the gradient is truly custom.

Pattern 3: Border Styling

Use `Modifier.border(width = PopStroke.Default, color = BorderColor.X, shape = shape)` or `Modifier.popBorder(width, gradient, shape)` for gradient borders.

Pattern 4: Icon Resolution

`Icons.X` (type-safe name) + `IconStyle.Outline/Solid` ->
`PopIconResources.getIconResourceId(iconName, style)` -> drawable resource ID. For buttons/visual elements: `VisualElement.buildFrom(Icons.X, IconStyle.Outline, fallbackResId = R.drawable.ic_xxx)`.

Pattern 5: Glow & Shadow Effects

`Modifier.glowEffect(blurRadius, spreadRadius, color, shape)` for standard glow.
`Modifier.intenseGlowEffect()` for stronger effects. These are in `com.pop.components.utils`.

Pattern 6: SquircleShape for Figma Corner Smoothing

When Figma uses 'Corner Smoothing' (smooth rounded rectangles), use `SquircleShape(cornerRadius, smoothing)` instead of `RoundedCornerShape`.

Pattern 7: Debounced Clicks

Use `rememberDebounceClick(onClick)` to prevent double-tap issues. This is used across `PopButtonV2` and other interactive components.

10. Preview / Testing System

Every major component has a corresponding preview file in the preview/ directory. These files contain @Preview annotated composables that showcase all variants, states, and sizes. The AI can use these as reference implementations.

Preview File Mapping

Component	Preview File	Previews Count	Key Variants Shown
PopButtonV2	PopButtonPreviews.kt	40+	All variants x states x sizes, loading states, icon combinations
PopInputFieldV2	PopInputFieldPreviews.kt	32	All input types, all status states, all configurations
PopOtpField	PopOtpFieldPreviews.kt	13	Different digit counts, states, error/success
PopChipStack	PopChipStackPreviews.kt	16	Chip arrangements, selection states
PopAccordion	PopAccordionPreviews.kt	25	Expanded/collapsed, different content
PopShimmer	PopShimmerPreviews.kt	27	All loader types and sizes
PopTitleBar	PopTitleBarPreviews.kt	19	Different title configurations
PopAvatar	PopAvatarPreviews.kt	9	All avatar types and sizes
PopCarousel	PopCarouselPreviews.kt	10	Different pagination styles, auto-scroll
PopTabLayout	PopTabLayoutPreviews.kt	9	Equal vs Wrap modes, icon tabs
PopChip	PopChipPreviews.kt	9	All variants and modes
PopBadge	PopBadgePreviews.kt	5	All badge types with/without bg/glow
PopBottomBar	PopBottomBarPreviews.kt	5	Different item counts, selection states
PopPills	PopPillsPreviews.kt	5	2-pill and 3-pill, with/without bg
PopOfferToggle	PopOfferTogglePreviews.kt	6	Offer toggle states and configurations
PopKeyPad	PopKeyPadPreviews.kt	4	Different keypad configurations
PopTypography	PopTypographyPreviews.kt	1	Complete typography showcase
PopVerticalButton	PopVerticalButtonPreviews.kt	4	Vertical button variants
PopListItem	PopListItemPreviews.kt	1	List item with various trailing content
PopSelectionControls	PopSelectionControlsPreviews.kt	1	Checkbox, Radio, Toggle together
PopTopBar	PopTopBarPreviews.kt	6	Different top bar configurations
PopHeroCarousel	PopHeroCarouselPreviews.kt	2	Hero carousel variants
PopOfferHighlight	PopOfferHighlightPreviews.kt	7	Highlight card variants

PopShimmerBlocks	PopShimmerBlocksPreviews.kt	4	Block-level shimmer layouts
Cards	CardPreviews.kt	3	Card layout examples

11. Utility & Effect Systems

11.1 Modifier Extensions (com.pop.components.utils)

Modifier Function	Purpose	Parameters
popBackground()	Apply gradient or solid background	gradient: PopGradient, shape: Shape
popBorder()	Apply gradient border	width: Dp, gradient: PopGradient?, shape: Shape
glowEffect()	Add glow/shadow effect	blurRadius: Dp, spreadRadius: Dp, color: Color, shape: Shape
intenseGlowEffect()	Stronger glow for brand elements	Similar to glowEffect with stronger defaults
noiseOverlay()	Add noise texture overlay	NoisePresets.* for predefined noise patterns
horizontalGradientFade()	Gradient fade at horizontal edges	Used in marquee text, carousel edges
bottomPrimaryGradientWithDither()	Gradient with dither for bottom bars	Used in PopBottomBar

11.2 Scrim / Blur System (ds_components/scrim/)

A 20-file subsystem adapted from the Haze library (Christopher Banes, Apache 2.0). Provides progressive blur effects for top bars, bottom sheets, and overlays.

File	Role
ScrimCore.kt	Core state management and API surface
ScrimEffect.kt	Effect abstraction
ScrimSource.kt	Blur source configuration
BlurModifierExtensions.kt (13KB)	Modifier.blurModifier() — primary integration point for components
ScrimBlurRenderEffect.kt (14KB)	Platform render effect implementation
ScrimEffectNode.kt (21KB)	Compose node for effect rendering — largest scrim file
ScrimBlurVisualEffect.kt	Visual effect layer
ScrimBlurColorEffect.kt	Color tinting for blur
ScrimBlurDelegates.kt	Delegate pattern for blur variants
ScrimProgressive.kt	Progressive blur (intensity varies with distance)
ScrimBlurDefaults.kt	Default configuration values

Key Usage Pattern:

```
val scrimState = rememberScrimState()
```

```
Modifier.blurModifier(scrimState) // Apply blur to content behind
```

12. Appendix: Full File Index

12.1 ds_components/ (58 files)

File Name	Size	Description
AlternatingSubtitleText.kt	3.4 KB	Alternating subtitle text animation
AvatarType.kt	6.0 KB	Sealed interface AvatarType + AvatarSize enum
ButtonLoadingState.kt	0.7 KB	Enum: Default, Success, Destructive, Intermediate
ButtonSpecs.kt	7.4 KB	Figma-exact button specifications object
ButtonState.kt	0.8 KB	Enum: Active, Disabled, Loading, Destructive
ButtonVariant.kt	0.8 KB	Enum: Primary, Secondary, Tertiary, BrandPrimary, White, Ghost, BrandGhost
EmptyStateComponent.kt	4.2 KB	Empty state: image + title + subtitle + button
EmptyStateComponentPreview.kt	1.9 KB	Preview for EmptyStateComponent
FavIcon.kt	2.7 KB	Favorite icon overlay
OverlappingAvatarIcons.kt	3.4 KB	Overlapping avatar group
PopAccordion.kt	13.7 KB	Expandable accordion with bounce animation
PopAppBar.kt	8.8 KB	App bar with navigation + action icon slots
PopAppBarDsl.kt	20.8 KB	DSL builder for app bar configuration
PopAppBarManager.kt	15.9 KB	Cross-screen app bar state management
PopAvatar.kt	16.8 KB	Universal avatar component
PopBadge.kt	6.8 KB	Badge with label, icons, bg, glow
PopBottomBar.kt	27.1 KB	Bottom navigation bar
PopBottomSheet.kt	42.8 KB	Modal bottom sheet
PopButtonV2.kt	40.1 KB	Primary button component
PopCarousel.kt	49.4 KB	Horizontal carousel with pagination
PopCheckBoxV2.kt	11.0 KB	Checkbox component
PopChip.kt	11.8 KB	Chip: Basic, WithClose, WithDropdown
PopChipStack.kt	9.7 KB	Chip container/arrangement
PopDot.kt	5.3 KB	Dot indicator
PopHeroCarousel.kt	29.0 KB	Full-width hero carousel
PopInputFieldHelpers.kt	7.5 KB	Input field helper functions
PopInputFieldV2.kt	108.3 KB	Multi-type input field (LARGEST)
PopKeyPad.kt	9.6 KB	Numeric keypad
PopKeyPadHelpers.kt	3.2 KB	Keypad helper functions
PopLazyColumn.kt	5.7 KB	Enhanced LazyColumn

PopLazyVerticalGrid.kt	9.2 KB	Enhanced LazyVerticalGrid
PopListItem.kt	23.4 KB	Generic list item
PopMarqueeText.kt	15.3 KB	Horizontally scrolling marquee text
PopOfferHighlight.kt	17.5 KB	Offer highlight card
PopOfferToggle.kt	10.3 KB	Offer toggle component
PopOtpField.kt	27.2 KB	OTP/PIN entry field
PopPills.kt	7.4 KB	Pill-shaped selectable group
PopRadio.kt	4.0 KB	Radio button
PopShimmer.kt	33.2 KB	Shimmer/skeleton loader
PopShimmerBlocks.kt	8.5 KB	Pre-composed shimmer blocks
PopStickyHeader.kt	11.1 KB	Sticky header for lists
PopStickyHeaderGrid.kt	5.5 KB	Sticky header for grids
PopTabLayout.kt	12.4 KB	Underline-style tab layout
PopTitleBar.kt	13.2 KB	Title bar sub-component
PopToastView.kt	10.6 KB	Toast notification
PopToggle.kt	11.3 KB	Toggle switch
PopTopBar.kt	22.0 KB	Complete top bar with collapsing
PopTopBarScrimType.kt	1.0 KB	Scrim type enum for top bar
PopTopBarScrollBehavior.kt	12.7 KB	Scroll behavior for top bar
PopTopBarWithContent.kt	49.5 KB	Extended top bar with content
PopTopBarWithCustomContent.kt	22.9 KB	Top bar with custom content
PopTopBarWithXmlContent.kt	21.4 KB	Top bar bridging XML content
PopVerticalButton.kt	7.2 KB	Vertical icon + label button
PopVisualElement.kt	11.7 KB	Universal visual element renderer
RiveAnimationCompose.kt	6.9 KB	Rive animation integration
RotatingText.kt	3.9 KB	Rotating text animation
Scrim.kt	14.0 KB	Top-level scrim composable
SquircleShape.kt	5.7 KB	Figma-style smooth rounded rectangle

12.2 scrim/ (20 files)

File Name	Size	Description
BlurModifierExtensions.kt	13.1 KB	Modifier extensions for blur effects — primary integration point
ScrimBlurColorEffect.kt	3.0 KB	Color tinting for blur effects
ScrimBlurDefaults.kt	1.8 KB	Default blur configuration values
ScrimBlurDelegates.kt	7.9 KB	Delegate pattern for blur variants
ScrimBlurDirtyFields.kt	2.2 KB	Dirty field tracking for optimization
ScrimBlurEffectScope.kt	0.8 KB	Effect scope for blur rendering
ScrimBlurHelpers.kt	4.9 KB	Helper functions for blur calculations
ScrimBlurRenderEffect.kt	14.4 KB	Platform render effect implementation
ScrimBlurRenderEffectDelegate.kt	7.3 KB	Render effect delegate
ScrimBlurVisualEffect.kt	9.6 KB	Visual effect layer for blur
ScrimCore.kt	3.5 KB	Core state management and API
ScrimEffect.kt	2.0 KB	Effect abstraction
ScrimEffectNode.kt	20.9 KB	Compose node for rendering — largest file
ScrimEffectScope.kt	1.3 KB	Effect scope
ScrimInputScale.kt	1.5 KB	Input scaling for blur
ScrimPlatformRenderEffect.kt	9.9 KB	Platform-specific render effect
ScrimProgressive.kt	6.5 KB	Progressive blur implementation
ScrimSource.kt	8.7 KB	Blur source configuration
ScrimUtils.kt	2.9 KB	Utility functions for scrim system
ScrimVisualEffect.kt	5.2 KB	Visual effect abstraction

12.3 preview/ (26 files)

File Name	Size
CardPreviews.kt	6.9 KB
PopAccordionPreviews.kt	13.3 KB
PopAvatarPreviews.kt	11.8 KB
PopBadgePreviews.kt	6.6 KB
PopBottomBarPreviews.kt	7.9 KB
PopButtonPreviews.kt	31.3 KB
PopCarouselPreviews.kt	15.1 KB
PopChipPreviews.kt	12.5 KB
PopChipStackPreviews.kt	27.1 KB

PopEnterAmountHeaderPreviews.kt	1.4 KB
PopHeroCarouselPreviews.kt	8.0 KB
PopInputFieldPreviews.kt	75.6 KB
PopKeyPadPreviews.kt	7.9 KB
PopListItemPreviews.kt	7.5 KB
PopOfferHighlightPreviews.kt	14.8 KB
PopOfferTogglePreviews.kt	13.7 KB
PopOtpFieldPreviews.kt	31.9 KB
PopPillsPreviews.kt	11.7 KB
PopSelectionControlsPreviews.kt	10.0 KB
PopShimmerBlocksPreviews.kt	2.7 KB
PopShimmerPreviews.kt	13.1 KB
PopTabLayoutPreviews.kt	8.1 KB
PopTitleBarPreviews.kt	13.9 KB
PopTopBarPreviews.kt	5.3 KB
PopTypographyPreviews.kt	10.9 KB
PopVerticalButtonPreviews.kt	13.5 KB

— *End of Document* —

POP UI Design System Codebase Metadata v1.0
Generated for AI-Powered Figma-to-Code Automation